

미어캣 위원회 — 최종 발표 읽는 말 (15분)

★ 이건 오빠 연습용 사본이다. 멘토에게 보낸 PDF 에는 이 머리말이 없다.

이 시험이 어떻게 도나 (총 최대 40분)

언제	무엇
9/17 23:59	영상 + PPT 업로드 마감
9/18 14:00	AI 질의 생성 · 모의 시험도 이때 열린다 — 본 시험 전에 모의를 먼저 본다
9/18 14:00 ~ 3일	편한 시간에 카메라 켜고 15분 발표(이 글을 읽어도 된다) → 질문 5개, 질의 당 최대 5분

★ 발표는 읽어도 되지만 답변은 못 읽는다. 카메라를 보며 말한다 — 08-질문_답변카드.md 는 글이 아니라 세 마디 뼈대로 외운다. ★ 짧게 답하고 넘긴다. 5분을 다 쓰면 안 물은 구멍까지 내가 연다.

시계 확인 지점 — 이 시각에 이 장을 시작하고 있으면 정상

시각	시작할 장
3:00	5 · 전체 흐름
5:15	7 · 첫 번째 벽: 입력
8:17	11 · AI가 일하는 자리 일곱
10:13	13 · 성과: 숫자
11:31	14 · 무엇이 막고 있는가
11:46	15 · 흔들림이 실제로 어떤 모습인가
13:09	16 · 배운 것과 남은 것

★ 밀렸을 때 빨 곳 — 위에서부터 뺀다 (합쳐 약 25초)

지금 그대로 읽으면 14분 57초다. 15분에 거의 붙어 있다 — 늘어지면 바로 넘는다. 1번과 2번은 미리 빼고 읽는 걸 권한다 — 그러면 14분 32초쯤 된다.

- 1. 슬라이드 11 「요즘 말로 하면 Constitutional AI 와 닮아 있습니다…」 문단 — 약 13초
- 2. 슬라이드 13 「그런데 이 대목에서 질문을 많이 받았습시다…」 문단 — 약 12초

★ 5 · 13 · 15 장은 절대 줄이지 않는다. 전체 흐름 · 숫자 · 흔들림이 발표의 기둥이고 질문도 거기서 나온다. ★ 새로 들어온 14장(무엇이 막고 있는가)은 한 문장으로 짧게 — 숫자 읽고 바로 넘긴다.

읽는 요령

- [슬라이드 N - …] 은 넘기는 자리 표시다. 읽지 않는다.
- 굵은 글씨는 세게 읽는다. 숫자는 「세 개」 · 「두 개」 로 읽는다 — 「셋」 은 말로 하면 안 들린다.
- 총 5697자 ≈ 14분 57초. 위의 「빨 곳」 을 꼭 먼저 보고 들어간다.

[슬라이드 1 - 표지] 0:00

안녕하세요. 미어캣 위원회를 만든 정기백입니다.
설계와 판단은 제가 하고, 구현은 AI 에이전트 여럿을 동시에 돌려서 했습니다. 밤에도 작업이 이어지게 자동화를 만들어 두고, 아침에 결과를 받아 방향을 다시 정하는 식으로 두 달을 보냈습니다. 결과물보다 그 과정이 더 재미있었습니다.
오늘 드릴 이야기는 하나입니다. 더 똑똑한 AI를 만든 이야기가 아니라, AI에게 무엇을 맡기고 무엇을 맡기지 않을지를 어떻게 정했는가입니다.

[슬라이드 2 - 문제] 0:29

저는 금융 리스크 시스템 쪽 일을 오래 했습니다. 회사에서는 한도를 넘으면 시스템이 막습니다. 그 담당자의 기분과 상관없이요. 그런데 저를 포함한 일반 개인에게는 한도를 관리해 주는 리스크 시스템이 없다는 걸 깨달았습니다.
개인 각자에게 규칙이 없어서가 아닙니다. 규칙은 다들 갖고 있습니다. 문제는 그 규칙이 필요한 순간이 하필 판단력이 제일 낮은 순간이라는 겁니다.
규율은 세울 때가 아니라 흔들릴 때 필요합니다.
그래서 이걸 직접 만들어 보고 싶었습니다.

[슬라이드 3 - 만든 것] 0:57

밤 열한 시. 십만 원짜리를 장바구니에 넣고 결제 버튼 앞입니다. 앱을 엽니다. “이거 사도 될까.”

“이번 달 벌써 두 번째예요. 지난번에 산 것과 겹치고, 스스로 정하신 규칙은 십만 원 넘으면 사를 기다리기였어요.” 그리고 선택은 사용자가 합니다.

챗봇과의 차이는 하나입니다. 챗봇의 답은 그 자리에서 지어낸 답입니다. 이걸 그 사용자가 멀쩡할 때 적어둔 원칙 문장과 대조한 결과입니다. 그래서 내일 같은 상황을 물어도 같은 답이 나옵니다.

그리고 하나 더 합니다. 전에 심의받은 내용과 그때 남긴 말을 모아 둔 것이 있습니다. 말뭉치, 코퍼스라고 부릅니다. 지금 상황과 닮은 대목을 찾아서 그 자리에 불러옵니다. 낱말이 겹치는 것을 찾는 게 아니라 뜻이 가까운 것을 찾습니다 — 문장을 숫자 벡터로 바꿔 두고 가까운 것을 고르는, 요즘 말로 검색 증강 방식입니다.

“지난달에도 같은 자리에서 이번엔 다르다고 하셨습니다. 그때 사를 기다리고 안 사셨고요.”

그때의 내가 지금의 나를 한 번 더 붙잡아 주는 겁니다.

대신 더 많이 물어봅니다. 판정에 필요한 항목이 다 차야 검사가 되니까요. 그 교환이 이 제품의 성격입니다. 그리고 그 되묻는 자리가 이 프로젝트에서 제일 어려웠던 곳입니다. 조금 뒤에 그 이야기를 하겠습니다.

[슬라이드 4 - 구성: 엔진과 팩] 2:11

구성은 크게 두 가지입니다. 엔진 하나, 그리고 팩. 출발은 이 두 문장이었습니다. 판정을 코드로 두면 답이 흔들리지 않는다. 분야를 팩으로 떼어 내면 엔진을 다시 안 만들어도 된다.

엔진은 도메인을 모릅니다. 연금이 원지 모릅니다. 아는 건 「원칙과 대조한다」는 절차뿐입니다. 분야가 아는 것은 전부 팩에 있습니다 — 받을 항목, 낱말, 검사할 원칙.

팩은 설정 파일입니다. YAML 로 씁니다. 분야를 넓힐 때 고치는 건 이 파일뿐이고, 엔진은 그대로 재활용합니다.

지금 팩이 세 개 있습니다. 연금, 대출, 충동구매. 새 분야를 붙일 때 엔진 코드는 한 줄도 안 고쳤습니다.

오늘 보여드릴 대표 팩은 연금투자 팩입니다. 제일 오래 돌려 봤고, 원칙을 어기는 방식이 제일 다양합니다. 시연 영상에는 대출 팩과 충동구매 팩도 같이 찍어 두었습니다.

[슬라이드 5 - 전체 흐름] 3:00

안을 통째로 보여드리겠습니다. 이 그림은 색깔이 곧 설계입니다.

파란 실선이 코드입니다. 검사하고 확정합니다. 빨간 점선 안이 AI입니다 — 위원 다섯이고, 읽고 말로 옮기기만 합니다. 초록은 선언 파일입니다. 새 분야를 열 때 고치는 건 여기뿐입니다.

사용자 문장이 왼쪽 위로 들어와 접수층을 지나고, 원칙 검사는 파란 칸 안에서 끝납니다. 그다음에야 위원들이 그 판정을 말로 품니다.

빨간 칸 안에 제가 이렇게 적어 뒀습니다 — “숫자를 다시 계산하지 않는다.” 코드가 낸 판정을 AI가 되돌리는 길이 없다는 뜻입니다.

이 그림 한 장이 오늘 드릴 이야기의 전부이고, 다음 장부터는 이 그림의 칸을 하나씩 확대하는 것입니다 — 파랑과 빨강이 합쳐 엔진입니다.

[슬라이드 6 - 계약: 모델이 읽고, 코드가 검사한다] 3:41

이 발표에서 제일 중요한 것을 먼저 말씀드립니다. 뒤가 전부 여기에 걸려 있습니다.

판정은 코드가 합니다. LLM 모델은 두 곳에만 씁니다 — 사람 말을 읽는 자리, 결과를 사람 말로 푸는 자리.

이유는 한 문장입니다. 읽기는 무한하지만, 검사는 유한합니다.

제가 만든 프로그램에서 정한 계약은 이렇습니다. 항목마다 모델이 읽을 것과 코드가 검사할 것을 한 쌍으로 선언합니다. 모델에게는 값의 모양과 설명을 주고, 코드에는 검사 함수 하나를 담니다. 모델은 읽기만 합니다. 확정은 언제나 검사 함수 안에서만 납니다.

항목 하나가 실제로 이렇게 생겼습니다. 모델에게 줄 것 두 가지 — 값의 모양과, 자연어 설명. 코드가 가질 것 두 가지 — 검사 함수와, 되읽어 줄 문장을 만드는 함수. 네 칸이 한 묶음입니다. 새 항목을 늘린다는 건 이 네 칸을 하나 더 선언한다는 뜻이고, 엔진은 그걸 그냥 돌립니다.

그리고 거절 사유가 세 가지뿐입니다. 모양이 안 맞거나, 정해진 목록에 없거나, 사용자가 쓴 말과 다르거나. 세 번째가 핵심입니다 — 모델이 지어낸 값을 잡는 자리입니다. 값을 다시 문장에 대보고, 그 자리에 실제로 없으면 거절합니다.

세 가지뿐이니까 시험지로 잠글 수 있습니다. 시험지란 정답을 사람이 먼저 확정해 둔 문제 묶음입니다. 저는 이걸 골든이라고 부릅니다. 코드를 고칠 때마다 전 문항을 다시 채점합니다. 한 문항이라도 틀리면 그 변경은 못 들어옵니다.

이 시험지로 팩 갈아끼우기도 했습니다. 대출 팩을 처음 붙였을 때 열 문항 중 다섯만 맞았습니다. 정답지도 엔진도 안 건드리고, 팩 설정만 고쳐서 전부 맞았습니다.

[슬라이드 7 - 첫 번째 벽: 입력] 5:15

그럼 이 계약은 어디서 나왔을까요. 벽 두 개에서 나왔습니다.

가설은 단순했습니다. 엔진 하나에 팩만 갈아 끼운다. 이 갈래는 맞았습니다. 입력에서 막혔습니다.

판정하려면 알아야 합니다. 무엇을, 얼마나, 사는지 파는지, 있던 돈인지 새 돈인지. 분야마다 더 붙습니다.

입력 양식으로 만들었더니 아무도 안 쓸 물건이 됐습니다. 사고 싶어 급한데 빈칸을 채우라고 하면 안 씁니다. 그래서 채팅으로 바꿨습니다.

[슬라이드 8 - 두 번째 벽: 만 줄짜리 낱말표] 5:43

그랬더니 사람 말을 알아듣는 코드가 자라기 시작했습니다.

“매도”만 알면 될 줄 알았습니다. 사람들은 “팔고”, “정리하려고”, “빼려는데”, “덜어내고” 이렇게 씁니다. 못 알아들을 때마다 한 줄씩 더하다 그 코드가 만 줄이 됐습니다.

진짜 문제는 끝이 없다는 겁니다. 내일 아침에도 못 읽는 문장이 나옵니다. 파서를 앞세우고 AI를 보조로 쓰는 한, 이길 수 없는 싸움입니다.

그래서 순서를 뒤집었습니다. 조금 전에 말씀드린 계약이 바로 이 벽에서 나왔습니다.

규율 하나를 얻었습니다. 낱말을 목록에 더해서 증상을 없애지 마라. 증상이 사라지면 고친 것 같지만 다음 증상을 미룬 것뿐입니다.

[슬라이드 9 - 접수증] 6:23

아까 그림에서 왼쪽 위 첫 번째 파란 칸입니다. 그 계약이 사는 자리를 접수증이라고 부릅니다. 말을 받아서 판정이 쓸 수 있는 값으로 만드는 증입니다.

전에는 낱말 목록이 먼저 읽고 못 읽으면 AI가 도왔습니다. 지금은 반대입니다.

이 층은 떼어낼 수 있는 부품으로 설계했습니다. 이유가 있습니다. 자기 규율 앱 자체는 시장이 크지 않습니다. 그런데 이 부품은 회사에서 업무로 쓰는 앱에도 그대로 재활용할 수 있습니다. 사람 말을 받아 정해진 항목으로 바꾸는 일은 어느 업무에나 있으니까요.

그래서 경계를 세게 그었습니다. 화면이 쓰는 렌더링 도구도, 서버도, 어느 분야인지도 모릅니다. 그 경계를 말이 아니라 자동 검사가 막습니다 — 여기서 시험이 빨개집니다. 이 부품만 시험 육백 건이 됩니다.

[슬라이드 10 - 터미널 로그: 층 일곱] 7:08

그 칸이 실제로 도는 모습입니다. 로그로 보시겠습니다 — 결과 화면보다 이쪽이 정확합니다.

먼저 하나 짚습니다. 이 시스템은 사라 팔라를 말하지 않습니다. 사용자가 정해 둔 원칙과 대조만 합니다.

층이 일곱 개입니다. 문장을 쪼개고, 조각을 항목에 배분하고, 검사하고, 애매하면 가르고, 모자라면 되묻고, 확정된 값을 되읽어 주고, 마지막에 판정 입력을 봉투에 담습니다.

줄 앞에 누가 처리했는지가 찍힙니다. 첫 층만 AI입니다. 나머지 여섯은 전부 코드입니다.

여기서 중요한 건 AI가 값을 확정하지 않는다는 겁니다. AI는 “이 조각이 어느 항목을 가리키는지” 이름표만 붙입니다. 값을 채우는 건 코드입니다. 나중에 필터로 막는 게 아니라, 애초에 판단할 권한을 주지 않습니다.

검사에서 막히면 되묻습니다. AI에게 다시 시키지 않고 사람에게 묻습니다. 답하면 그 항목에만 꽃힙니다. 자기 교정을 모델 안이 아니라 사람과의 왕복에 뒀습니다. 후보를 벌려놓고 좁히는 구조라 생각의 나무 — Tree of Thoughts 의 얇은 버전으로 보셔도 됩니다. 다만 나무를 모델 안에서 키우지 않고 밖에서 코드가 가지를 칩니다.

[슬라이드 11 - AI가 일하는 자리 일곱] 8:17

그림 가운데 빨간 점선 안 위편 다섯이 여기입니다. 그럼 AI는 어디 쓰나. 여기까지만 들으면 “AI를 별로 안 썼네”로 들릴 수 있어서 넣은 장입니다. 일곱 군데입니다.

첫째, 방금 본 문장 쪼개는 자리.

둘째부터 여섯째가 위편 다섯 명입니다. 시장분석가는 외부 데이터를 읽어 옵니다 — 연금 팩은 한국거래소 종목 데이터를, 총동구매 팩은 공공데이터를 붙입니다. 원칙검사관은 코드가 낸 판정을 말로 풀고, 과거기록관은 지난 심의를 인용하고, 의장은 정리하되 결론을 내지 않습니다. 반론자만 다른 위원의 말을 읽고 반박합니다.

전부 대화하게 만들까도 했습니다. 안 했습니다. 다섯 명이 의논해서 합의에 이르면, 그 합의가 곧 추천이 됩니다. 서술은 대화여도 되지만 판정이 대화의 산물이 되면 안 됩니다.

요즘 말로 하면 Constitutional AI 와 닿아 있습니다. 사람이 써 둔 원칙 문장을 헌법처럼 두고 거기에 비추는 구조니까요. 다만 저는 그 헌법을 모델에게 읽히지 않았습니다. 코드가 검사합니다.

일곱째가 이 제품을 만든 공장입니다. 판단 자리에 상위 모델 하나, 일하는 자리에 여러 모델. 작업 공간 네 개를 병렬로 돌리고, 일이 끝나면 스스로 다음 일을 큐에 넣습니다. 감시 프로세스가 오 분마다 확인하고 죽었으면 갈아끼웁니다. 돌린 작업 묶음이 구백 번을 넘었습니다.

제품 안에서는 AI의 자율성을 억제했고, 만드는 과정에서 코딩 공장 자동화는 극단적으로 풀었습니다.

안 쓰는 곳은 딱 하나. 판정입니다.

[슬라이드 12 - 왜 랭그래프가 아니었나] 9:45

설계 초기에 랭그래프·랭체인을 붙일지 검토했고, 안 쓰기로 했습니다.

랭류즈 같은 관측 도구를 붙이면 모델이 언제 무슨 판단을 했는지 다 따라갈 수 있습니다. 하지만 제가 필요했던 건 추적이지 아니라 재현이었습니다. 같은 입력에 같은 판정이 나오는가. 그건 어떤 도구를 붙여도 모델 안에서는 보장되지 않습니다.

그래서 체인이 아니라 계약을 골랐습니다.

[슬라이드 13 - 성과: 숫자] 10:13

숫자로 말씀드립니다.

먼저 왜 판정을 코드로 뒀는가. 팔월 오일에 모델 게이트웨이에 직접 확인했습니다. 추론 모델은 온도 파라미터를 아예 받지 않습니다. 영으로 뒀는데 안 먹는 게 아니라 줄 자리가 없습니다. 난수 씨앗은요? 문서가 “최선을 다한다”고만 적혀 있습니다.

그런데 이 대목에서 질문을 많이 받았습니다. “요즘 모델은 잘하지 않냐”, “온도를 영으로 두면 되지 않냐”. 문서 인용으로는 설득이 안 됐습니다. 그래서 썼습니다.

문장 백여순여섯 개를 각각 세 번에서 다섯 번씩. 팔월 이십구일 첫 측정, 오백열 번 중 스물아홉 건이 매번 다른 답을 뱉습니다. 어제 오늘 코드로 다시 썼습니다. 오백마흔여섯 번 중 서른한 건. 두 달 사이 모델도 코드도 바뀌었는데 줄지 않았습니다.

코드 쪽은 켈 필요가 없습니다. 같은 입력이면 같은 출력이니깐요. 대신 맞는지를 썼습니다. 원칙마다 검사 함수를 따로 짜던 것을 규칙을 선언하면 그대로 도는 방식으로 다시 썼습니다 — 판정 엔진 1에서 2입니다. 같은 정답지를 옛 코드와 새 코드에 넣고 대조했더니 서른여덟 건 중 여긔난 개 하나

도 없었습니다.

두 숫자는 다른 것을 잰 것입니다. 앞은 같은 답이 나오는가, 뒤는 맞는 답이 나오는가.

재현되지 않는 판정은 판정이 아닙니다.

[슬라이드 14 - 무엇이 막고 있는가] 11:31

숫자 하나만 더 보여드립니다. 이 모든 걸 지키는 자동 검사가 접수층에만 육백 건, 서버 사전오백 건, 웹 삼천오백오십 건 됩니다. 하나라도 빨개지면 그 변경은 못 들어갑니다.

[슬라이드 15 - 흔들림이 실제로 어떤 모습인가] 11:46

숫자만으로는 안 와닿으실 테니 실제로 어떻게 흔들리는지 보여드립니다. 전부 제 말뭉치의 진짜 문장입니다.

“보너스 전부 때려박을까.” 어떤 회차는 아무 항목도 못 읽고, 어떤 회차는 금액과 방향을 모두 읽습니다. 같은 문장입니다.

“반도체가 빠진 김에 오백만 원 더 넣겠습니다. 세 번에 구입 일로 나눠서요.” 물타기입니다. 금액과 분할 계획을 같이 말하는데 회차마다 읽는 항목이 달라집니다.

“어제 산 거 또 살까.” 이런 판정에 들어가는 값 자체가 세 가지로 같았습니다.

그래서 어제 한 겹 더 잰습니다. 판정 입력을 통째로 해시해서 회차끼리 비교했습니다. 흔들린 서론한 건 중 열여섯 건은 그 흔들림이 검사층에서 걸러지지 않고 판정 입력까지 그대로 갔습니다.

여기서 배운 게 있습니다. 결정론 코드층은 비결정론 AI층의 흔들림을 눌러 주지 않습니다. 흡수도 증폭도 안 하고 그대로 통과시킵니다.

그래서 흔들림은 줄이는 게 아니라 드러내는 것으로 정리했습니다. 감추면 사용자는 확정된 답으로 읽습니다. 드러내면 한 번 더 확인할 기회가 생깁니다.

그리고 가드레일을 한 겹 더 넓혀야 한다는 것도 여기서 나왔습니다. 지금 검사는 세 가지뿐이고, 값 하나하나가 진짜인지만 봅니다. 값들이 제자리에 갖는지는 안 봅니다. “A 팔고 B 산다”에서 A와 B를 바꿔 끼워도 세 가지 검사를 모두 통과합니다. 한 덩어리로 말한 값들이 서로 다른 항목으로 흩어지는 것을 막는 검사, 그게 다음에 추가 개발할 부분입니다.

[슬라이드 16 - 배운 것과 남은 것] 13:09

배운 것은 세 가지입니다.

첫째, “모른다”에 이름을 줘야 합니다. 처음엔 답이 두 가지였습니다. 위반이거나, 위반이 아니거나. 그런데 검사할 재료가 없어서 검사를 못 한 경우가 조용히 “위반 아님”으로 들어갑니다. 그래서 세 번째 답을 만들었습니다. “검사 대상 아님.” 대출에서 “주택담보대출”이라고만 하면 후보가 세 개인데 고를 근거가 없습니다. 아무거나 채우지 않고 비운 채 멈춥니다. 그럴듯한 값을 채우는 순간 사용자는 제대로 알아들었다고 믿어 버립니다.

둘째, 읽기는 무한하고 검사는 유한합니다.

셋째, 낱말을 더해서 증상을 없애지 마라. 증상이 살아 있으면 고친 층이 틀린 겁니다.

못 한 것은 두 가지입니다. 접수층 부품은 이미 만들었고, 화면까지 신호를 끝까지 잇는 일이 남았습니다. 부품 잘못이 아니라 연결을 마저 잇는 일입니다.

그리고 방금 그 열여섯 건. 값 하나하나가 진짜인지는 검사하는데, 값들이 제자리에 찍지어 갖는지는 다음 가드레일로 넣을 계획입니다.

마지막으로 하고 싶은 것. 지금은 새 분야를 열려면 제가 설정 파일을 손으로 씁니다. 좀 더 고도화해서 상용화하려면 팩 스튜디오를 추가로 개발해야겠다는 생각을 하고 있습니다. 회사가 내부 규정을 말로 설명하면, 말투나 도메인에 맞는 정보를 설정하는 야를 파일도, 그릇 역할을 하는 DB 스키마 항목 구조도 AI가 만들어 주는 도구입니다.

이 도구, 팩 스튜디오야말로 이 주제를 한 번 더 증명하는 자리입니다. 규정을 읽는 것은 무한하지만, 지켜졌는지 검사하는 방법은 유한하니까요. 다시 말해 사람 말로 자기 도메인 이야기를 풀어내면, 야물과 스키마도 만들어 주니까요.

AI에게 무엇을 맡기고 무엇을 맡기지 않을지, 저는 감이 아니라 실측으로 정했습니다. 그리고 그 실측 자체가 이 프로젝트에서 AI를 가장 깊이 쓴 자리였습니다.

들어주셔서 감사합니다.

— 14:57 끝 —